Maximum Edge-based Quasi-Clique: Novel Iterative Frameworks

Hongbo Xia, Shengxin Liu, Zhaoquan Gu

Harbin Institute of Technology, Shenzhen, China
{24s151167@stu., sxliu@, guzhaoquan@}hit.edu.cn

***Abstract*—Extracting cohesive subgraphs from complex networks is a fundamental task in graph data management and is essential for understanding biological, social, and technological systems. The edge-based γ-quasi-clique model provides a flexible alternative by identifying subgraphs whose edge densities exceed a specified threshold γ. However, identifying the exact maximum edge-based quasi-clique is computationally challenging, as the problem is NP-hard and lacks the hereditary property. These characteristics limit the effectiveness of conventional pruning techniques and hinder the development of efficient reduction rules. As a result, existing algorithms, such as `QClique` and `FPCE`, suffer from scalability issues when applied to large networks. In this paper, we propose a novel iterative framework that reformulates the problem as a sequence of hereditary subproblems, enabling more effective pruning and reduction strategies, and improving the worst-case time complexity. Furthermore, we redesign the iterative process to improve theoretical efficiency and introduce a new heuristic strategy to guide the search more effectively. Extensive experiments on 253 large-scale real-world graphs demonstrate that our proposed algorithm `EQC-Pro` outperforms existing methods by up to four orders of magnitude.**

***Index Terms*—Cohesive subgraph mining, edge-based quasi-clique, iterative framework**

I. Introduction

Many types of real-world information can be naturally represented as graphs, including biological protein networks, social networks, and web graphs. A fundamental problem in this context is the extraction of *cohesive subgraphs*. However, due to noise and incomplete edge information, the strict definition of a clique (i.e., a pairwise fully connected subgraph) is often too restrictive for practical scenarios. To address this limitation, various relaxed clique models have been proposed, such as the k-plex [44], k-defective clique [55], k-core [43], and k-truss [50]. Collectively, these are referred to as cohesive subgraph or clique relaxation models.

One intuitive relaxation is the edge-based γ-quasi-clique [3], [2], which introduces a parameter (between 0 and 1) to control the degree of relaxation. Specifically, a subgraph is considered an *edge-based γ-quasi-clique* if the ratio between the number of edges it contains and the number of edges in a complete graph of the same size is at least γ. When $\gamma = 1$, the definition reduces to the classical clique model. Recent studies [40] have provided strong empirical evidence that the edge-based γ-quasi-clique model offers practical benefits in biological network analysis. For instance, compared to the degree-based γ-quasi-clique model – which requires each vertex in the subgraph to be connected to at least a γ-fraction of the other vertices – the edge-based model achieves better predictive performance in identifying protein complexes, as measured by sensitivity (SNS), precision (PPV), and geometric accuracy [36]. Moreover, compared to heuristic approaches [36], clusters obtained from exact solutions exhibit higher biological relevance, as demonstrated by Gene Ontology enrichment[1] analysis, which reveals enrichment in more unique and functionally meaningful cellular components. These results highlight the practical advantages of exact dense subgraph mining for uncovering biologically significant structures. Beyond biological networks, the edge-based quasi-clique model has also proven effective in a variety of other contexts, including the analysis of social networks [5], [19] and the detection of anomalies [46], [56].

In this paper, we revisit the problem of finding the exact maximum edge-based quasi-clique [49], [41], [34]. Specifically, given an undirected graph and a density threshold γ, our goal is to identify the largest edge-based γ-quasi-clique. This problem is challenging because it has been proven to be NP-hard [39], which creates substantial computational difficulties for exact solutions. Moreover, the edge-based γ-quasi-clique does not have the hereditary property; that is, its subgraphs are not necessarily edge-based γ-quasi-cliques. Consequently, many advanced techniques that have been developed for finding cohesive subgraphs with the hereditary property, such as k-plexes [11], [12], [22] and k-defective cliques [8], [17], [9], cannot be directly applied to this problem. This limitation reduces the effectiveness of traditional pruning strategies and further increases the complexity of the problem.

State-of-the-Art Algorithms. Existing exact algorithms for the maximum edge-based quasi-clique problem can be broadly categorized into two main approaches: mixed integer programming (MIP) and branch-and-bound techniques. While MIP-based methods [39], [34], [49] can solve instances with up to 10^4 vertices, they often struggle to scale to larger graphs in practice. Consequently, recent research has primarily focused on developing efficient branch-and-bound algorithms.

Among these, `QClique` [41] stands out as the most effective exact branch-and-bound algorithm. It builds upon the branching strategy of `PCE` [48], which leverages the quasi-hereditary structure of edge-based quasi-cliques [39].

[1]https://geneontology.org/

QClique introduces a new upper bound for pruning, and achieves performance competitive with both earlier branch-and-bound and MIP-based approaches. In contrast, FPCE [40] represents the most advanced maximal enumeration algorithm and incorporates several refined pruning strategies. Although originally designed for maximal edge-based quasi-cliques enumeration, FPCE can be adapted to solve the exact maximum problem by dynamically updating the current largest edge-based quasi-clique during the search.

Despite these advances, both QClique and FPCE share certain limitations. Specifically, they rely on the same branching mechanism as PCE, making their pruning strategies inherently dependent on this structure. This reliance makes it difficult to design effective reduction rules for further efficiency improvements; indeed, no such reduction rules have been proposed in the literature to date. As a result, the overall efficiency of these solvers remains constrained, especially for large-scale graphs where graph reduction is crucial.

Our New Methods. We reformulate the problem as a sequence of hereditary subproblems, enabling effective pruning and reduction techniques that leverage hereditary properties. With this reformulation, we not only formally prove the correctness of the proposed iterative framework under the edge-based quasi-clique model, but also achieve the worst-case time complexity of $O^*(\beta_\kappa^n)$, where O^* suppresses polynomial factors, n is the number of vertices, $\beta_\kappa < 2$ is the largest real root of $x^{\kappa+2} - 2x^{\kappa+1} + 1 = 0$, and κ relates to the size of the optimum solution. Compared with existing exact algorithms such as QClique and FPCE, both of which have $O^*(2^n)$ complexity, our approach yields theoretical improvements.

Moreover, by leveraging the quasi-hereditary property of the edge-based quasi-clique, we develop a bottom-up doubling iterative framework that reduces the number of iterations required to solve hereditary subproblems from $O(n)$ to $O(\log s^*)$ in theory, where s^* denotes the size of the optimum solution, which is typically much smaller than n in practice. In addition, we design a simple yet effective heuristic based on the degeneracy sequence, as commonly adopted in cohesive subgraph mining. To address the inflexibility caused by over-reliance on this sequence, we introduce a dynamically refined heuristic strategy that iteratively updates the candidate solution by adding or removing vertices. This strategy diversifies selection process and may lead to better heuristic solutions.

Contributions. Our contributions are summarized as follows.

- We adapt the top-down iterative framework to the edge-based quasi-clique setting by reducing the problem to a sequence of hereditary subproblems, thereby achieving improved complexity of $O^*(\beta_\kappa^n)$. (Section III)
- By leveraging the quasi-hereditary property of edge-based quasi-clique, we propose a bottom-up doubling iterative framework that reduces the number of iterations required to solve hereditary subproblems from $O(n)$ to $O(\log s^*)$. (Section IV)
- We design a novel heuristic strategy, further improving practical performance. (Section V)

We conduct extensive experiments on 253 graph datasets from two benchmark collections, with up to 5.92×10^7 vertices (Section VII). Experimental results show that our proposed EQC-Pro achieves up to four orders of magnitude speedup over state-of-the-art baselines QClique and FPCE.

II. Preliminaries

Let $G = (V, E)$ be an undirected simple graph with $|V| = n$ vertices and $|E| = m$ edges. We denote $G[S]$ of G as the subgraph induced by the set of vertices $S \subseteq V$. For any $S \subseteq V$ and $u \in V$, let $d_G^S(u)$ denote the number of neighbors of u in S, i.e., $d_G^S(u) = |\{v \in S \mid v \neq u \text{ and } (u, v) \in E\}|$. Moreover, we use $N_G[u]$ to denote the closed neighborhood of vertex u in G, i.e., $N_G[u] = \{v \in V \mid v = u \text{ or } (u, v) \in E\}$, and $N_G[S]$ to denote the union of $N_G[u]$ for all vertices $u \in S$, i.e., $N_G[S] = \bigcup_{u \in S} N_G[u]$. For an induced graph g of G, the vertex set and the edge set of g are denoted by $V(g)$ and $E(g)$, respectively. For a subgraph g, we define *the number of missing edges* as the number of edges absent in g compared to a complete graph of the same size, i.e., $\binom{|V(g)|}{2} - |E(g)|$. In this paper, we focus on the edge-based γ-quasi-clique (γ-EQC) [3], [2], which is defined below.

Definition 1. *Given a real number $0 < \gamma < 1$, a graph g is said to be an* edge-based γ-quasi-clique *(γ-EQC) if we have $|E(g)| \geq \gamma \times \binom{|V(g)|}{2}$.*

In other words, an edge-based γ-quasi-clique is an induced subgraph in which the number of edges is at least a γ faction of the number of edges in the corresponding complete graph, i.e., $\binom{|V(g)|}{2}$. We are ready to present our problem in this paper.

Problem 1 (Maximum edge-based γ-quasi-clique)**.** *Given a graph $G = (V, E)$ and a real number $\gamma \in [0.5, 1)$, the Maximum Edge-based γ-Quasi-Clique Problem (MaxEQC) aims to find an edge-based γ-quasi-clique in G with the largest number of vertices.*

Let g^* and $s^* = |V(g^*)|$ be the maximum γ-EQC and its size. It is known that finding the maximum γ-EQC is NP-hard for any fixed $\gamma \in (0, 1)$ [39]. Note that, unlike cliques, γ-EQCs lack the well-known *hereditary property* [39]: not all subgraphs of a γ-EQC are guaranteed to remain γ-EQCs. This limitation poses challenges for efficient algorithmic design, as even checking maximality by inclusion becomes a non-trivial task for γ-EQC. Nevertheless, γ-EQCs satisfy a weaker structural guarantee, known as the *quasi-hereditary property*, which plays a crucial role in our algorithmic design.

Proposition 1 (Quasi-hereditary property [39])**.** *Let $G = (V, E)$ be a graph, and $S \subseteq V$ such that $G[S]$ is a γ-EQC. If there exists a vertex $v \in S$ with degree less than the average degree in the subgraph $G[S]$, i.e., $d_G^S(v) \leq \frac{2|E(G[S])|}{|S|}$, then $G[S']$, where $S' = S \setminus \{v\}$, is also a γ-EQC.*

Proposition 1 implies that for a given edge-based γ-quasi-clique g, removing a vertex satisfying a specific degree constraint guarantees that the remaining graph remains an edge-

based γ-quasi-clique, whereas removing an arbitrary vertex does not, due to the non-hereditary property of γ-EQCs.

III. A Top-down Iterative Framework

A closely related problem to MaxEQC is the *maximum degree-based quasi-clique problem (MaxQC)*, which has been effectively addressed by the state-of-the-art iterative algorithm IterQC [52]. However, it is important to note that neither of these two problems satisfies the hereditary property. Therefore, inspired by the key iterative idea behind IterQC, our first framework seeks to transform the MaxEQC problem into a sequence of subproblems that do possess the hereditary property, following a top-down approach.

We introduce the cohesive subgraph of k-defective clique, which serves as a key component in our iterative approach.

Definition 2 ([55]). *Given an integer $k \geq 1$, a graph g is said to be a* k-defective clique *if* $|E(g)| \geq \binom{|V(g)|}{2} - k$.

In other words, a k-defective clique g contains at most k missing edges. Building on the concept of k-defective clique, extensive studies [14], [21], [32], [8], [9], [18] in the literature have focused on the following problem.

Problem 2 (Maximum k-defective clique). *Given a graph $G = (V, E)$ and a positive integer k, the Maximum k-Defective-Clique Problem (kDC) aims to find the largest k-defective-clique in G.*

Before introducing the top-down iterative framework, we define the following two functions.

- $\texttt{get-k}(n) := \lfloor (1-\gamma) \cdot \binom{n}{2} \rfloor$, which takes a number n of vertices as input and returns an appropriate value of k;
- $\texttt{solve-defect}(k)$:= the size of the largest k-defective clique in G, which takes a value k as input.

Overview. The key observation is that *the maximum γ-EQC g^* in the graph G is also the maximum k^*-defective clique, where k^*=get-k(s^*)*, as directly implied by the definitions. Motivated by this, the top-down iterative framework decomposes the MaxEQC problem into a sequence of kDC problems. Since s^* is unknown, the framework starts with an upper bound on this size (e.g., the number of vertices in the graph, i.e., $|V|$) and iteratively solves the corresponding maximum k-defective clique problem, where k is determined by the current upper bound. At each iteration i, the maximum k_i-defective clique g_i is found. If g_i is not a valid γ-EQC (i.e., if $\texttt{get-k}(|V(g_i)|) < k_i$), then $|V(g_i)|$ (or equivalently, $\texttt{get-k}(|V(g_i)|)$) serves as an upper bound for s^* (or k^*), because otherwise g^* would be a k_i-defective clique larger than the maximum one g_i, which contradicts the maximality of g_i, given that $k^* \leq k_i$ and every k'-defective clique is also a k''-defective clique for $k' \leq k''$. The framework then updates the upper bound to this tighter value (i.e., $|V(g_i)|$) and repeats the process. This top-down iterative approach progressively narrows the search space until the largest γ-EQC is found.

Algorithm. Our top-down iterative framework (named as EQC-TD) is shown in Algorithm 1. Line 1 initializes an upper bound as s_0 via UpperBound (described later). This upper bound is iteratively refined within the while-loop (Lines 2-6). In each iteration, EQC-TD assumes the existence of a size-s_{i-1} γ-EQC and computes the corresponding upper bound on the number of missing edges, denoted as $k_i = \texttt{get-k}(s_{i-1})$. Using this value, EQC-TD invokes solve-defect to obtain the maximum size s_i of a k_i-defective clique. We show that this procedure provides a tighter upper bound for the optimal solution s^* (see the appendix of [1]). The algorithm terminates when the k-sequence converges, i.e., when $k_i = \texttt{get-k}(s_i)$ in Line 4, and returns the corresponding size s_i as the final result in Line 5. The correctness proof of our top-down iterative framework (Algorithm 1) is deferred to the appendix of [1].

Algorithm 1: A Top-down Iterative Framework for MaxEQC: EQC-TD

Input: A graph $G = (V, E)$ and a real value $0 < \gamma < 1$
Output: The size of the maximum γ-EQC in G

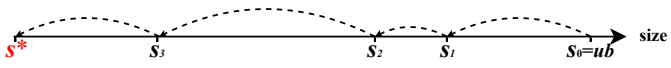
1 $s_0 \leftarrow \texttt{UpperBound}(G)$; $k_1 \leftarrow \texttt{get-k}(s_0)$; $i \leftarrow 1$;
2 **while** *true* **do**
3 $s_i \leftarrow \texttt{solve-defect}(k_i)$;
4 **if** $k_i = \texttt{get-k}(s_i)$ **then**
5 $s^* \leftarrow s_i$; **return** s^*;
6 $k_{i+1} \leftarrow \texttt{get-k}(s_i)$; $i \leftarrow i + 1$;

Algorithm 2: UpperBound(G)

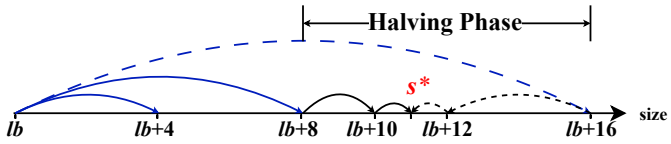
Output: An upper bound ub of s^*

1 $ub \leftarrow (1 + \sqrt{1 + 8m/\gamma})/2$; $k \leftarrow \texttt{get-k}(ub)$;
2 **while** *true* **do**
3 $ub \leftarrow \texttt{ComputeUB}(k)$;
4 **if** $\texttt{get-k}(ub) \geq k$ **then break**;
5 $k \leftarrow \texttt{get-k}(ub)$;
6 **return** ub;

Upper Bound. An upper bound is commonly used in the literature on the MaxEQC problem [49], [39], which is used to estimate the size of the maximum γ-EQC by solving $m \geq \gamma \cdot \binom{ub}{2}$, yielding $ub \leq (1+\sqrt{1+8m/\gamma})/2$. However, this bound is often overly coarse. To obtain a tighter upper bound, we incorporate the coloring-based upper bounding strategy from the kDC problem [8], [18] into *an iterative framework tailored for the γ-EQC setting*. Our method for computing upper bounds for the MaxEQC problem is based on a key observation. In our top-down iterative framework (Algorithm 1), we maintain upper bounds for the optimal values $k^* = \texttt{get-k}(s^*)$ and s^* at each iteration. By consistently choosing k and s as upper bounds, each iteration produces progressively tighter upper bounds. A formal proof of this property is provided in Appendix of [1]. Thus, *if the k value in some iteration is an upper bound of k^*, then the upper bound computed for the corresponding kDC problem is also an upper bound of s^**. Based on this observation, we summarize our upper bound computation method UpperBound in (Algorithm 2),



(a) The top-down iterative strategy (Algortihm 1)



(b) The bottom-up doubling iterative strategy (Algorithm 3)

Fig. 1: Illustration of iterative frameworks.

where ComputeUB is an upper bound estimation procedure for the kDC problem [8], [18]. Specifically, UpperBound first initializes a trivial ub as $(1+\sqrt{1+8m/\gamma})/2$ and sets the corresponding value of k in Line 1. Then, UpperBound iteratively invokes ComputeUB with different values of k (Lines 2-6 of Algorithm 2) until the condition $\texttt{get-k}(ub) \geq k$ is satisfied in Line 4. Finally, we return ub as the upper bound of s^* in Line 6. For completeness, we provide the details of ComputeUB in Appendix of [1].

Example. Figure 1(a) illustrates the top-down iterative strategy, in which the algorithm repeatedly attempts to tighten the upper bound on the solution size. Dashed arrows represent these successive updates, which continue until the bound converges to the optimal size s^*. However, since each step does not guarantee a fixed reduction, the rate of progress may vary considerably across iterations. For example, there may be a substantial decrease from s_2 to s_3, or only a negligible improvement from s_1 to s_2.

Limitations. Our top-down iterative algorithm EQC-TD (Algorithm 1) can successfully reduce the MaxEQC problem – which lacks the hereditary property – into multiple instances of the kDC problem, which does possess this property. However, this framework still suffers from the following efficiency issues. **First**, in the worst case, EQC-TD may require up to n calls to solve-defect (in Line 3 of Algorithm 1), leading to excessive number of iterations. **Second**, although kDC is hereditary, its computational cost becomes prohibitive as k increases. Specifically, for a fixed density threshold γ, the get-k function indicates that k grows quadratically with the number of vertices. While ComputeUB provides a tighter upper bound, the lack of a generally efficient and tight estimation forces the top-down strategy to start with a large s^*, resulting in excessive iterations and degraded performance.

IV. An Advanced Bottom-up Iterative Framework

To overcome the limitations of the top-down iterative framework in Section III, we leverage the unique structural property of the γ-EQC and introduce a specially tailored *bottom-up* iterative framework in this section. It is important to note that our bottom-up iterative framework is specifically designed for the MaxEQC problem and cannot be directly applied to the problem related to degree-based quasi-cliques. Before presenting our bottom-up framework, we first introduce an important property of the iterative framework for γ-EQC, which is derived from Proposition 1.

Proposition 2. *Recall that s^* denote the size of the maximum edge-based γ-quasi-clique. Then for any $s \leq s^*$, it holds that*

$$s \leq \texttt{solve-defect}(\texttt{get-k}(s)),$$

and for any $s > s^*$, it holds that

$$s > \texttt{solve-defect}(\texttt{get-k}(s)).$$

Proof. We first recall that the function $\texttt{solve-defect}(k)$ returns the *largest* integer s' such that there exists a k-defective clique of size s' in G. In the following, we consider two cases according to the relationship between s and s^*. In both cases, we let $k = \texttt{get-k}(s) = \lfloor (1-\gamma)\binom{s}{2} \rfloor$.

Case 1: $s \leq s^*$. We show that it is always possible to construct a γ-EQC of size s from the maximum γ-EQC g^* of size s^*, such that $s \leq \texttt{solve-defect}(\texttt{get-k}(s))$ holds. Specifically, we begin with the maximum γ-EQC g^* of size s^* and iteratively remove a vertex of minimum degree from the subgraph until only s vertices remain. Since a vertex of minimum degree in the subgraph always has less than or equal to the average degree, the quasi-hereditary property (Proposition 1) ensures that the induced subgraph g on the remaining s vertices remains a γ-EQC. Thus, the number of edges in g is at least $\gamma \times \binom{s}{2}$, which implies that g is a feasible k-defective clique of size s since the number of missing edges is $\lfloor \binom{s}{2} - \gamma \times \binom{s}{2} \rfloor = k$. Consequently, we have $\texttt{solve-defect}(k) \geq s$.

Case 2: $s > s^*$. We prove by contradiction. Suppose, to the contrary, that $s \leq \texttt{solve-defect}(\texttt{get-k}(s))$. Let $s' = \texttt{solve-defect}(\texttt{get-k}(s))$, and let g' be the corresponding k-defective clique returned. By definition of a k-defective clique (Definition 2), the number of edges in g' satisfies $E(g') \geq \binom{s'}{2} - \texttt{get-k}(s)$. Since $\texttt{get-k}(s) \leq \texttt{get-k}(s') = \left\lfloor (1-\gamma)\binom{s'}{2} \right\rfloor$, we have $E(g') \geq \binom{s'}{2} - \left\lfloor (1-\gamma)\binom{s'}{2} \right\rfloor \geq \gamma\binom{s'}{2}$. Thus, g' corresponds to a γ-EQC of size s', and we know $s' \geq s > s^*$, which contradicts the maximality of s^*.

This completes the proof by considering both cases. □

Overview. Proposition 2 implies that we can identify the optimal solution s^* by analyzing the relationship between s and $\texttt{solve-defect}(\texttt{get-k}(s))$. In particular, based on Proposition 2, we can devise a more efficient iterative framework by performing a binary search on the candidate size s. Consequently, the number of iterations to determine s^* can be reduced from $O(n)$ to $O(\log n)$, thereby improving efficiency. Nevertheless, it is important to note that the efficiency of $\texttt{solve-defect}(k)$ is highly dependent on the value of k. Specifically, the problem becomes easier to solve as k decreases. This observation motivates us to explore alternative search strategies that can further reduce the computational cost, especially in the early stages of the iterative framework.

Algorithm 3: A Bottom-up Doubling Framework for MaxEQC: EQC−BU

Input: A graph $G = (V, E)$ and a real value $0 < \gamma < 1$
Output: The size of the maximum γ-EQC in G

1 $lb \leftarrow 1$; $ub \leftarrow |V|$;
// Doubling Phase
2 $gap \leftarrow 1$;
3 **while** $lb + gap \leq ub$ **and** *check*$(lb + gap)$ **do**
4 $\quad lb_{\text{new}} \leftarrow lb + gap$;
5 $\quad gap \leftarrow 2 \times gap$;
// Halving Phase
6 $ub_{\text{new}} \leftarrow \min(lb + gap - 1, ub)$;
7 **while** $lb_{new} \neq ub_{new}$ **do**
8 $\quad mid \leftarrow \lceil (lb_{\text{new}} + ub_{\text{new}})/2 \rceil$;
9 $\quad$ **if** *check*(mid) **then**
10 $\quad\quad lb_{\text{new}} \leftarrow mid$;
11 $\quad$ **else**
12 $\quad\quad ub_{\text{new}} \leftarrow mid - 1$;
13 **return** lb_{new};
14 **Function check**(s)**:**
15 $\quad k \leftarrow$ get-k(s);
16 $\quad s' \leftarrow$ solve-defect(k);
17 $\quad$ **return** $s' \geq s$;

To this end, we make use of a *bottom-up doubling strategy* to progressively approach the optimal value s^*. Specifically, let lb and ub denote the current lower and upper bounds of s^*. The standard binary search approach would iteratively select the midpoint, $s = \lfloor (lb + ub)/2 \rfloor$, as the candidate value of s to evaluate at each iteration, updating the bounds based on the result. While this method is efficient in terms of the number of iterations, it may not always be efficient in practice, particularly when we need to invoke solve-defect with a large value of k (e.g., when ub is extremely large).

In contrast, our bottom-up doubling strategy begins by probing the values of s in an exponentially increasing sequence $\{lb + 1, lb + 2, lb + 4, \dots\}$, doubling the step size at each iteration until the value exceeds s^*. After a candidate interval that must contain s^* is identified, we then switch to a more localized search, i.e., we perform a binary search within this narrowed interval. This approach keeps the values of k encountered in the early rounds as small as possible, which can potentially improve the overall efficiency of the bottom-up doubling strategy.

Algorithm. Our proposed bottom-up doubling iterative framework EQC−BU is detailed in Algorithm 3. EQC−BU efficiently search for the optimal value s^* by dynamically adjusting the lower bound lb and upper bound ub of s^*. In Line 1, EQC−BU initializes lb to 1 and ub to $|V|$, as it holds that $1 \leq s^* \leq |V|$. The core idea of EQC−BU is to efficiently and progressively identify the search interval containing s^* via the check subroutine (described in Lines 14-17). This subroutine evaluates whether a candidate value of s does not exceed the optimal solution size s^*, as guaranteed by Proposition 2. The evaluation process consists of two phases: a doubling phase and a halving phase.

1) **Doubling Phase (Lines 2-5)**. The doubling phase first initializes the step size gap to 1 (Line 2) and iteratively calls check$(lb + gap)$ for increasing values of gap if $lb + gap$ does not exceed the current upper bound ub (Line 3). If check returns true, then by Proposition 2, $lb + gap$ is feasible, which implies $lb + gap \leq s^*$. This value is then used to update the lower bound lb_{new} (Line 4), which will be the starting point for the halving phase. Subsequently, in Line 5, the algorithm doubles gap to expand the interval upward, aiming to quickly approach or exceed s^*. This process repeats until either the feasibility check fails or the upper bound ub is reached in Line 3. Note that if the current candidate value of $lb + gap$ is not feasible, we know that $s^* < lb + gap$, which will be used to update ub_{new} in the halving phase.
2) **Halving Phase (Lines 6-12)**. After the doubling phase terminates, the algorithm proceeds to a binary search (halving) phase to precisely locate the optimal value s^*. In this phase, the refined interval $[lb_{\text{new}}, ub_{\text{new}}]$ is set according to the final state after doubling (Line 6). The halving phase then performs a standard binary search within this narrowed interval. In each iteration, it tests the midpoint $mid = \lceil (lb_{\text{new}} + ub_{\text{new}})/2 \rceil$. If check$(mid)$ returns true, the lower bound lb_{new} is updated to mid; otherwise, the upper bound ub_{new} is set to $mid - 1$ (Lines 7-12). This binary refinement continues until $lb_{\text{new}} = ub_{\text{new}}$, at which point the optimal value s^* is identified.

Finally, EQC−BU returns lb_{new} as s^* in Line 13.

Example. To provide a clearer comparison with the top-down iterative framework, Figure 1 shows the top-down and bottom-up strategies. Dashed arrows indicate updates to the upper bound, while solid arrows represent improvements to the lower bound. The bottom-up doubling approach (Figure 1(b)) begins by rapidly identifying a solution interval through the doubling phase (shown in blue), and then performs the having phase (shown in black) within that interval to locate the optimal solution. Both approaches require $O(\log s^*)$ iterations in the worst case, where s^* denotes the size of the optimal solution.

Correctness and Remarks. We first remark that our EQC−BU consists of a doubling phase, which quickly finds an upper bound of s^*, followed by a halving phase that efficiently narrows the candidate interval containing s^*. EQC−BU is specifically tailored for our iterative framework for the MaxEQC problem, where the computation performed by the invoked kDC solver becomes increasingly costly as k grows.

For the correctness of EQC−BU (Algorithm 3), we observe that, by Proposition 2, check(s) is monotonic in s: once it fails at some s, it fails for all larger values; conversely, once it succeeds, it succeeds for all smaller values. After the doubling phase (Lines 2-5), we observe that the optimal value s^* always

Algorithm 4: `Degen-Opt`(G, γ)

Output: A large γ-EQC in G

1 $g \leftarrow$ `Degen`(G, γ);
2 **foreach** *vertex* $u \in V$ **do**
3 $\quad g' \leftarrow$ `Degen`$(G[N_G[u]], \gamma)$;
4 $\quad$ **if** $|V(g')| > |V(g)|$ **then** $g \leftarrow g'$;
5 **return** g;
6 **Function** `Degen`$(G,\ \gamma)$ **:**
7 $\quad$ Compute a degeneracy ordering in G;
8 $\quad$ $H \leftarrow$ the longest suffix of the degeneracy ordering such that $G[H]$ is a γ-EQC;
9 $\quad$ **return** $G[H]$;

lies in the interval $[lb_{\text{new}}, ub_{\text{new}}]$. This is because (1) lb_{new} is only increased when a feasible candidate is found in Lines 2-5, and (2) $ub_{\text{new}} = \min(lb + gap - 1, ub)$ is set in Line 6 when $lb + gap > ub$ or an infeasible candidate is detected in Line 3. We then enter the halving (i.e., binary search) phase on this narrowed interval. In each iteration of the binary search, the midpoint test either raises the lower bound or lowers the upper bound while preserving the invariant. The process terminates precisely when the two bounds coincide, at which point the common value must be s^*. Thus, `EQC-BU` is correct.

We then analyze the number of calls to `check` in `EQC-BU` (and will discuss the time complexity in Section VI). The doubling phase (Lines 2-5) performs at most $\lceil \log_2 s^* \rceil + 1$ calls to `check`, since the algorithm starts from a small candidate size (e.g., 1) and doubles the size until it exceeds the actual optimal size s^*. Once a valid upper bound is detected, where this upper bound is clearly at least s^* and at most $2s^*$, the doubling phase terminates. In the subsequent halving phase (Lines 6-12), the search interval lies strictly below $2s^*$ and is halved in each iteration. Thus, the number of iterations in this phase is also bounded by $\lceil \log_2 s^* \rceil$. Overall, the total number of calls to `check` is bounded by $O(\log s^*)$.

V. Our Novel Heuristic Strategies

Since our bottom-up doubling framework `EQC-BU` (Section IV) begins the iterations from a lower bound, i.e., a value that is smaller than the optimal value s^*, obtaining a larger lower bound can effectively reduce the number of iterations required by the framework. In this section, we propose novel heuristic strategies to compute a large lower bound for `MaxEQC`.

A. Basic Heuristic Methods: `Degen` and `Degen-opt`.

We first present two basic heuristic algorithms in Algorithm 4, both adapted from the heuristic method in `KDC` [8]. These algorithms utilize the concept of the *degeneracy ordering*, which is obtained by iteratively removing the vertex with the smallest degree from the graph [4]. The first basic heuristic algorithm `Degen` (shown in Lines 6-9 of Algorithm 4) iteratively removes the vertex with the minimum degree until the remaining subgraph forms a γ-EQC. However, `Degen` suffers from a structural limitation: once a vertex is selected into the solution, *all subsequent vertices in the degeneracy ordering are inevitably included as well*. This inflexible selection mechanism results in a rigid combination process, thereby restricting the ability of the algorithm to explore more promising candidate subgraphs. To address this issue, `Degen-opt` in Algorithm 4 partially relaxes this restriction. Specifically, for each vertex u in the graph, we consider the subgraph induced by its closed neighborhood, i.e., $N_G[u] = \{v \in V \mid v = u \text{ or } (u, v) \in E\}$, and execute `Degen` on this subgraph. In this way, each vertex has more opportunities to be included in the solution. Nevertheless, while this approach expands the combination space by constructing multiple sequences and thereby allows more flexibility in vertex selection, it still fundamentally relies on fixed suffixes of degeneracy sequences. Moreover, since `Degen-Opt` is applied only to subgraphs induced by closed neighborhoods, the diameter of any resulting solution is at most 2, which is not a necessary condition for edge-based quasi-cliques.

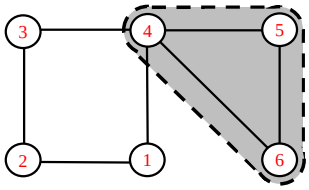


Fig. 2: An example with $\gamma = 0.75$, illustrating limitations of `degen` and `degen-Opt`.

Example. We illustrate the solution process of different heuristic methods using the structure shown in Figure 2, where the objective is to find the maximum 0.75-EQC in the given graph. The optimal solution has a size of 3, corresponding to the shaded area in the figure.

The construction of the degeneracy sequence proceeds by iteratively removing the vertex with the minimum degree. However, when degeneracy-based heuristics are applied to the entire graph, they do not necessarily guarantee a heuristic solution of size 3. For instance, the first vertex to be removed could be any from the set $\{1, 2, 3, 5, 6\}$, each having degree 2. As discussed previously, once a vertex is included in a candidate solution, all subsequent vertices later in the degeneracy sequence (i.e., those removed later) must also be included. If the first vertex removed is from the subset $\{5, 6\}$, the heuristic solution cannot reach size 3, as the only 0.75-EQC of size 3 must include precisely the three specific vertices. In contrast, `degen-Opt` is guaranteed to obtain a solution of size 3 in this case, as it applies the degeneracy-based strategy to the ego-network of each vertex. For example, for vertex 6, its ego-network contains vertices $\{4, 5, 6\}$, and the last three vertices in the corresponding degeneracy sequence must be exactly these three, forming a valid 0.75-EQC. However, this example also demonstrates a fundamental limitation of the method: it still relies on suffixes of degeneracy orderings, and the resulting heuristic solution is inherently constrained to have

Algorithm 5: EQC-Heu(G, γ, g)

Input: A graph $G = (V, E)$, a real value $0 < \gamma < 1$, and a heuristic solution g
Output: An improved γ-EQC in G

1 $g' \leftarrow g; k \leftarrow$ get-k$(|V(g')| + 1)$;
2 **foreach** *vertex* $u \in V(g)$ **do**
3 $\quad S \leftarrow \{u\}$; $C \leftarrow V \setminus \{u\}$;
4 $\quad$ **while** $|E(G[S])| \geq \binom{|S|}{2} - k$ **do**
5 $\quad\quad$ **if** $|S| > |V(g')|$ **then**
6 $\quad\quad\quad g' \leftarrow G[S]$; $k \leftarrow$ get-k$(|V(g')| + 1)$;
7 $\quad\quad$ AddBestVertex(S, C);
8 $\quad\quad$ AddBestVertex(S, C);
9 $\quad\quad$ RemoveWorstVertex(S, C);
10 $\quad\quad$ UpdateScore(S, C);
11 **return** g';
12 **Function** AddBestVertex(S, C)**:**
13 $\quad P \leftarrow \{u \in C \mid d_G^S(u) = \max_{v \in C} d_G^S(v)\}$;
14 $\quad P' \leftarrow \{u \in P \mid \text{score}(u) = \max_{v \in P} \text{score}(v)\}$;
15 $\quad u \leftarrow$ a randomly chosen vertex from P';
16 $\quad$ move u from C to S;
17 $\quad$ score$[u] \leftarrow 0$;
18 **Function** RemoveWorstVertex(S, C)**:**
19 $\quad P \leftarrow \{u \in S \mid d_G^S(u) = \min_{v \in S} d_G^S(v)\}$;
20 $\quad P' \leftarrow \{u \in P \mid \text{score}(u) = \min_{v \in P} \text{score}(v)\}$;
21 $\quad u \leftarrow$ a randomly chosen vertex from P';
22 $\quad$ move u from S to C;
23 $\quad$ score$[u] \leftarrow 0$;
24 **Function** UpdateScore(S, C)**:**
25 $\quad$ **foreach** *vertex* $v \in S$ **do**
26 $\quad\quad$ score$[v] \leftarrow$ score$[v] + d_G^C(v) - \Delta_G$;
27 $\quad$ **foreach** *vertex* $v \in C$ **do**
28 $\quad\quad$ score$[v] \leftarrow$ score$[v] + d_G^S(v)$;

a diameter of at most two due to the ego-network construction.

B. Our Dynamic Refined Heuristic Strategies

To further mitigate the inflexibility caused by the combination of Degen-Opt, we proposed our heuristic methods, which *dynamically refine the heuristic solution by iteratively removing and adding vertices*. This strategy is designed to overcome the constraints imposed by degeneracy ordering on candidate solution construction.

Overview. Our dynamic refined heuristic strategies take as input a feasible γ-EQC g and iteratively adjust its vertex set to potentially obtain an improved heuristic solution. The core idea is to dynamically add and remove vertices between the selected set S and the candidate set C, while maintaining the γ-EQC property throughout the process. Specifically, for each vertex in g, we use it as a starting point (i.e., a *seed*) and initialize the selected set with this seed. At each iteration, we update the selected set by adding two vertices and removing one vertex, guided by a score function that aims to maximize the quality of the solution. After every update, we verify that the subgraph induced by the selected set remains a γ-EQC. This iterative process proceeds until the number of missing edges in the subgraph induced by the current selected set exceeds the allowable threshold. In particular, given the largest γ-EQC g' identified so far, we compute the maximum number of missing edges permitted for a γ-EQC of size $|V(g')| + 1$, since our goal is to find a larger one. After evaluating every vertex in g as a seed, we select the largest γ-EQC found during this process as our final solution.

The score function is designed to maximize the size of the heuristic solution by prioritizing the addition of vertices that increase the density of the subgraph induced by the selected set and the removal of those that detract from it. We use the scoring function from [13], where all vertex scores are initially set to zero. Whenever a vertex v is added to or removed from the selected set S, its score is reset to zero. After each iteration, if $v \in S$, its score decreases by $\Delta_G - d_G^C(v)$, where Δ_G is the maximum degree in G and $d_G^C(v)$ is the number of v's neighbors in the candidate set C. If $v \in C$, its score increases by $d_G^S(v)$, the number of neighbors in S.

EQC-Heu. Based on the above design, our heuristic strategy EQC-Heu is summarized in Algorithm 5. We first initialize the current largest γ-EQC g' and the maximum number k of missing edges permitted for a γ-EQC of size $|V(g')| + 1$ in Line 1 and consider each vertex u in $V(g)$ as a seed in Lines 2-10. For each u, we initialize the selected set $S \leftarrow \{u\}$ and the candidate set $C \leftarrow V \setminus S$ in Line 3. We then iteratively update S and C by adding two vertices from C to S (Lines 7–8) and removing one vertex from S to C (Line 9), using AddBestVertex (Lines 12–17) and RemoveWorstVertex (Lines 18–23), respectively. This process is guided by the score function UpdateScore (Lines 24–28). Importantly, after every update, we ensure that the subgraph induced by S continues to satisfy the γ-EQC property (Line 4). Once a larger γ-EQC is found, we update g' accordingly in Lines 5-6. The process repeats until the number of missing edges in the current selected set exceeds the allowable threshold, which is recalculated based on the size of the largest γ-EQC found so far. Finally, after all vertices in $V(g)$ are considered as seeds, we return the largest γ-EQC found (Line 11) as our solution. An illustrative example of the solving process of EQC-Heu is provided in Appendix of [1].

Improved EQC-Heu-Pro. Although EQC-Heu alleviates the issue of inflexible vertex combinations, applying it directly to the entire graph is inefficient. For large-scale graphs, maintaining set-related information incurs substantial memory overhead, and the associated random access patterns degrade cache performance, thereby reducing overall efficiency. To address these challenges, we propose an improved EQC-Heu-Pro, which applies the heuristic to multiple smaller subgraphs rather than the entire graph, as shown in Algorithm 6.

Specifically, the algorithm first extracts the subgraph induced by the closed neighborhood $N_G[V(g)]$ of the current solution g (Line 2), and then applies EQC-Heu to this induced

Algorithm 6: EQC-Heu-Pro(G, γ, g)

Input: A graph $G = (V, E)$, a real value $0 < \gamma < 1$, and a heuristic solution g
Output: An improved γ-EQC in G

1 **while** *true* **do**
2 $g' \leftarrow$ EQC-Heu$(G[N_G[V(g)]], \gamma, g)$;
3 **if** $|V(g')| = |V(g)|$ **then break**;
4 $g \leftarrow g'$;
5 **return** g;

subgraph to obtain a refined solution g'. If no improvement in size is achieved (i.e., $|V(g')| = |V(g)|$), the process terminates (Line 3); otherwise, g is updated to g' and the procedure is repeated (Line 4). This iterative refinement ensures efficiency and addresses the problem of inflexible solution construction without compromising solution quality.

Time Complexity Analyses. We analyze the time complexities of EQC-Heu (Algorithm 5) and EQC-Heu-Pro (Algorithm 6). First, in Algorithm 4, the initial call to Degen on the whole graph G requires $O(m)$ time [4]. Then, for each vertex $u \in V$, we extract the induced subgraph $G[N_G[u]]$ of size $|N_G[u]| = d_G(u) + 1$. On this subgraph, computing a degeneracy ordering and finding a γ-quasi-clique via Degen takes $O(d_G(u)^2)$ time [4]. Summing over all vertices yields

$$\sum_{u \in V} O(d_G(u)^2) \;\le\; \Delta_G \sum_{u \in V} d_G(u) \;=\; 2\,\Delta_G\, m,$$

where Δ_G is the maximum degree in G. Thus, the time complexity of Algorithm 4 is $O(m + \Delta_G\, m) = O(\Delta_G\, m)$.

We next analyze EQC-Heu in Algorithm 5, where the most expensive operations are the vertex addition or removal and the score updates in Lines 7–10. By maintaining, for each update, only the vertices adjacent to the currently selected set S, each such round of updates can be implemented in $O(\Delta_G\,|S|) = O(\Delta_G\, s^*)$ time, since $|S| \le s^*$. The while-loop (Lines 4-10) increases the size of S by one in each iteration, and thus executes at most s^* times for each initial vertex u. The for-each-loop (Lines 2-10) over $u \in V(g)$ iterates at most $|V(g)| \le s^*$ times. Combining these factors, the total running time of Algorithm 5 is $O\big(|S| \times s^* \times \Delta_G s^*\big) = O\big(\Delta_G\,(s^*)^3\big)$.

For EQC-Heu-Pro in Algorithm 6, it is easy to see that there are at most s^* iterations in the while-loop (Lines 1–4). In each iteration, EQC-Heu-Pro invokes EQC-Heu once. Thus, the complexity of EQC-Heu-Pro is $O\big(\Delta_G\,(s^*)^4\big)$.

VI. Our Final Iterative Algorithm: EQC-Pro

Building on the insights from the iterative framework (Section IV) and heuristic strategies (Section V), we introduce our improved iterative algorithm EQC-Pro in this section. Before presenting EQC-Pro in detail, we first introduce important observations that can further improve practical performance.

Observation 1. *Additional useful information can be extracted from* solve-defect(k). *Specifically, for the* check *function in Algorithm 3, if it returns* true, *we can directly construct a solution to the* kDC *problem. Furthermore, since* get-k$(\cdot)$ *is a floor function, it follows that different values of s may correspond to the same value of k. Consequently, solutions to the* kDC *problem for one value of s may be reused for other values of s that yield the same k.*

Algorithm 7: Our Final Algorithm: EQC-Pro

Input: A graph $G = (V, E)$ and a real value $0 < \gamma < 1$
Output: The size of the maximum γ-EQC in G

1 **global** *memo* **as** map from k to s';
2 $g \leftarrow$ Degen-Opt(G, γ);
3 $g \leftarrow$ EQC-Heu-Pro(G, g, γ);
4 $gap \leftarrow 1$; $lb, lb_{new} \leftarrow |V(g)|$; $ub \leftarrow |V|$;
5 **while** $lb + gap \le ub$ **do**
6 $s \leftarrow lb + gap$; $k \leftarrow$ get-k(s);
7 $s' \leftarrow$ MemoizedSearch(k);
8 **if** $s' < s$ **then break**;
9 $lb_{new} \leftarrow s'$;
10 $gap \leftarrow 2 \times gap$;
11 $ub_{\text{new}} \leftarrow lb + gap - 1$;
12 **while** $lb_{new} \neq ub_{new}$ **do**
13 $mid \leftarrow \lceil (lb_{\text{new}} + ub_{\text{new}})/2 \rceil$; $k \leftarrow$ get-k(mid);
14 $s' \leftarrow$ MemoizedSearch(k);
15 **if** $s' \ge mid$ **then**
16 $lb_{\text{new}} \leftarrow mid$;
17 **else**
18 $ub_{\text{new}} \leftarrow mid - 1$;
19 **return** lb_{new};
20 **Function** MemoizedSearch(k)**:**
21 **if** $k \in$ *memo* **then**
  // memoization
22 $s' \leftarrow memo[k]$;
23 **else**
24 $s' \leftarrow$ solve-defect(k);
25 $memo[k] \leftarrow s'$;
26 $g \leftarrow$ the graph corresponding to the result returned from solve-defect(k) with size s';
27 $g \leftarrow$ EQC-Heu-Pro(G, g, γ); // expansion
28 **return** $|V(g')|$;

Observation 2. *Within a single iteration in Algorithm 3, the lower bound s_i obtained corresponds to the solution of the* kDC *problem for a particular value of k. However, since our ultimate objective is to determine the lower bound s^* for the* MaxEQC *problem, there remains potential for further improvement by leveraging this relationship.*

Building upon the two observations above, we are now ready to introduce our improved iterative algorithm EQC-Pro in Algorithm 7. Specifically, the design of EQC-Pro is based on the bottom-up doubling iterative framework EQC-BU (Algorithm 3) and the heuristic strategy EQC-Heu-Pro (Al-

gorithm 6). It further incorporates two key techniques in the MemoizedSearch function: *memoization* and *expansion*. To begin with, we initiate a global memoization table (Line 1), denoted as memo, which caches the results of previous calls to solve-defect(k) and thus avoids redundant computations across iterations. Subsequently, in Lines 2-3, we compute an initial heuristic solution g using the Degen-Opt algorithm and further refine it with EQC-Heu-Pro, which attempts to expand the solution. The size of the resulting solution is used to initialize the lower bound lb of the search space for the bottom-up iterative framework.

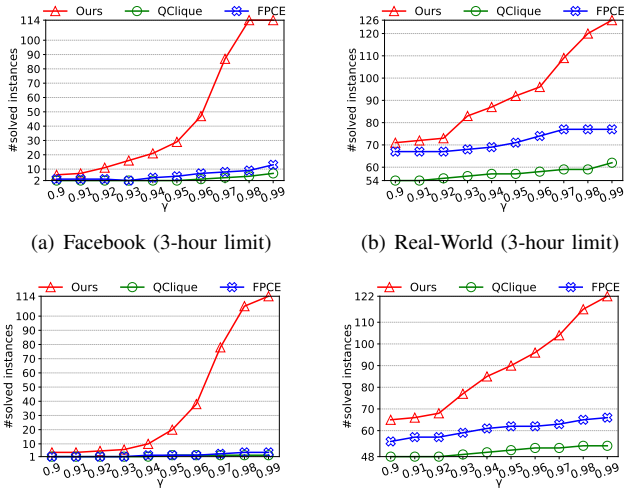
After the initialization in Line 4, EQC-Pro performs the bottom-up doubling iterative search in Lines 5-19 (which is similar to Algorithm 3), utilizing MemoizedSearch (described in Lines 20-28) in each iteration. Specifically, if the result of solve-defect for the current value of k has already been computed and cached in memo, we directly reuse the stored result; otherwise, we invoke solve-defect(k). If the size s' of the returned subgraph is at least s, we store this result in memo. Moreover, MemoizedSearch further *expands* the corresponding solution using EQC-Heu-Pro, which may yield a larger feasible solution. Once the iterative search terminates, the size of the maximum γ-EQC is returned in Line 19. The correctness follows from that of Algorithm 3.

Remarks and Time Complexity Analysis. We first remark that the get-k function can be implemented in a straightforward manner, while the solve-defect function can be implemented using existing state-of-the-art kDC algorithms [8], [9], [18]. In our implementation, we adopt the state-of-the-art algorithm **kDC-two** [9] for solve-defect.

For the time complexity of EQC-Pro (Algorithm 7), we note that EQC-Pro iteratively calls the MemoizedSearch function and, in essence, realizes the bottom-up doubling iterative framework EQC-BU described in Section IV. Similar to the discussion in that section, the total number of iterations of MemoizedSearch is bounded by $O(\log s^*)$. In each iteration of MemoizedSearch, the algorithm invokes solve-defect, which dominates the computational cost, while the accompanying heuristic procedure EQC-Heu-Pro runs in polynomial time and is invoked only once per iteration. The complexity of **kDC-two** (used for solve-defect in our implementation) is $O^*(\beta_k^n)$ [9], where β_k is the largest real root of the equation $x^{k+2} - 2x^{k+1} + 1 = 0$, and O^* omits polynomial factors. Note that the complexity of solve-defect increases monotonically with k. According to our analysis for EQC-UB in Section IV, the largest k that EQC-Pro needs to handle is $\kappa = \text{get-k}(2s^*)$. Thus, the overall time complexity of EQC-Pro is $O^*(\beta_\kappa^n)$, which is asymptotically better than the complexities of both QClique and FPCE, each of which has a complexity of $O^*(2^n)$.

VII. Experiments

Algorithms. We evaluate the efficiency of EQC-Pro compared to state-of-the-art exact algorithms for solving MaxEQC.



(a) Facebook (3-hour limit) (b) Real-World (3-hour limit)

(c) Facebook (300-second limit) (d) Real-World (300-second limit)

Fig. 3: Number of solved instances with varying γ.

- QClique[2]: the state-of-the-art branch-and-bound algorithm [41].
- FPCE[3]: a baseline adapted from the recent algorithm for enumerating all maximal γ-EQCs above a given size threshold [40]. We modify it for MaxEQC by maintaining the largest solution found so far and updating the lower bound accordingly to improve pruning.

We also evaluate the performance gains achieved by the proposed bottom-up framework (Section IV) and the proposed heuristic strategies (Section V).

- EQC-TD: Algorithm 1, the top-down iterative approach.
- EQC-NH: A variant of EQC-Pro that omits lower bound refinement using EQC-Heu-Pro by removing Lines 3 and 27 in Algorithm 7.

All algorithms are implemented in C++ and compiled using g++ with -O3 optimization. Experiments are conducted on a machine equipped with an Intel 2.60GHz CPU and 256GB of RAM, running Ubuntu 20.04.6. We enforce a maximum runtime of 3 hours (10,800 seconds) per instance.

Datasets. We conduct experiments on the following two collections, which are widely used in related studies.

- **Real-World Collection**[4]: 139 Real-World graphs from Network Repository, with up to 5.87×10^7 vertices.
- **Facebook Collection**[5]: 114 Facebook social networks, from Network Repository, with up to 5.92×10^7 vertices.

A. Comparison with Baselines

Number of Solved Instances. Figure 3 presents the number of instances solved by our proposed EQC-Pro algorithm and

[2]We re-implemented the algorithm based on the description in [41]. Our implementation achieves better performance compared to the results reported in their paper.

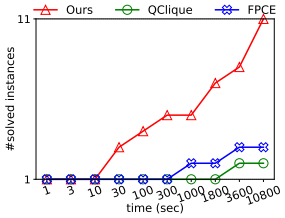
[3]https://github.com/ahsanur-research/FPCE

[4]http://lcs.ios.ac.cn/caisw/Resource/realworld%20graphs.tar.gz

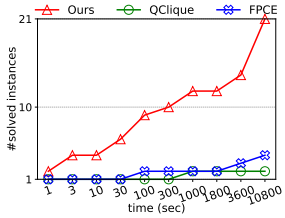
[5]https://networkrepository.com/socfb.php

TABLE I: Running time (in seconds) on 20 large-scale graphs with $\gamma = 0.95$.

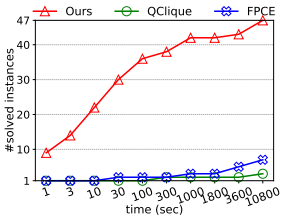
Graphs	EQC-Pro	EQC-TD	EQC-NH	QClique	FPCE	n	m	s^*
inf-road-usa	7.16	7.52	**5.93**	–	358.36	23M	57M	4
inf-roadNet-CA	0.57	0.62	**0.49**	–	3.58	1.9M	5M	4
web-wikipedia2009	**8.03**	–	39.10	–	–	1.8M	9M	34
soc-pokec	**205.40**	–	460.59	–	–	1.6M	44M	35
soc-lastfm	**19.66**	–	79.49	–	–	1.1M	9M	20
soc-youtube-snap	**6.88**	–	20.00	–	–	1.1M	5.9M	22
rt-retweet-crawl	1.44	**1.13**	1.26	–	11.93	1.1M	4.5M	16
inf-roadNet-PA	**0.31**	0.46	0.39	–	1.37	1M	3M	4
sc-ldoor	**211.23**	–	212.22	–	–	909K	41M	22
soc-delicious	**0.67**	–	4.64	–	–	536K	2.7M	30
soc-youtube	**4.31**	–	12.79	–	–	495K	3M	22
sc-msdoor	**101.36**	–	101.92	–	–	404K	18M	22
soc-twitter-follows	0.24	**0.16**	0.25	3335.43	5.28	404K	1.4M	7
ca-MathSciNet	**0.18**	–	0.19	–	158.14	332K	1.6M	25
sc-pwtk	**55.5**	–	100.45	–	–	217K	11M	26
socfb-konect	**22.82**	23.38	28.99	–	1902.99	59M	185M	7
socfb-uci-uni	**28.15**	29.59	33.39	–	–	58M	184M	7
socfb-A-anon	**6857.26**	–	–	–	–	3M	47M	37
socfb-B-anon	**1952.97**	–	–	–	–	2.9M	41M	35
socfb-Maine59	**2421.99**	–	3247.84	–	–	9K	486K	41



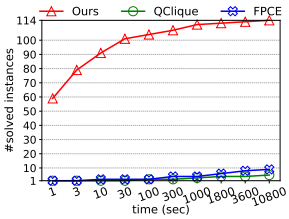
(a) $\gamma = 0.92$



(b) $\gamma = 0.94$

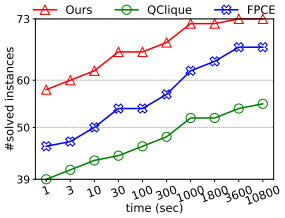


(c) $\gamma = 0.96$

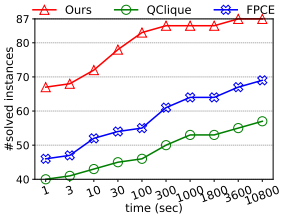


(d) $\gamma = 0.98$

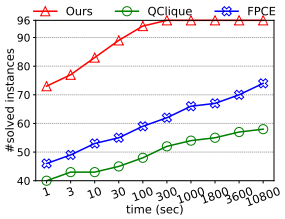
Fig. 4: Number of solved instances on Facebook.



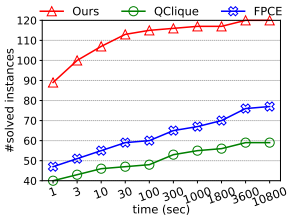
(a) $\gamma = 0.92$



(b) $\gamma = 0.94$



(c) $\gamma = 0.96$



(d) $\gamma = 0.98$

Fig. 5: Number of solved instances on Real-World.

two baseline methods, QClique and FPCE, under 3-hour and 300-second time limits, respectively, across two collections of datasets with varying γ values from 0.9 to 0.99. We have the following observations. **First**, the number of solved instances sharply decreases as γ becomes smaller. This is because a lower value of γ corresponds to a looser constraint on γ-EQC, which significantly weakens the effectiveness of pruning and reduction rules in branch-and-bound methods. Moreover, relaxed constraints typically lead to much larger solution sizes, thereby increasing the depth of the search and making the problem substantially more difficult to solve. **Second**, EQC-Pro consistently outperforms the two baselines across all tested values of γ. For example, in Figure 3(a), when $\gamma = 0.98$ on the Facebook collection, EQC-Pro solves all 114 instances, while QClique and FPCE only solve 5 and 9 instances, respectively. For the Real-World collection, at $\gamma = 0.9$ under the 300-second time limit, QClique and FPCE solve 48 and 55 instances, respectively, whereas EQC-Pro solves 65 instances as shown in Figure 3(d). The significant speedup achieved by EQC-Pro results from transforming the problem into a sequence of hereditary subproblems, which substantially accelerates the search process. Compared to the Real-World collections, the Facebook graphs are denser, making the problem more challenging to solve under the same values of γ. The advantage of EQC-Pro becomes more obvious at higher γ, where the problem is moderately constrained. However, as γ decreases, the search space increases dramatically, resulting in greater computational difficulty and reducing the performance gap between our method and the baselines.

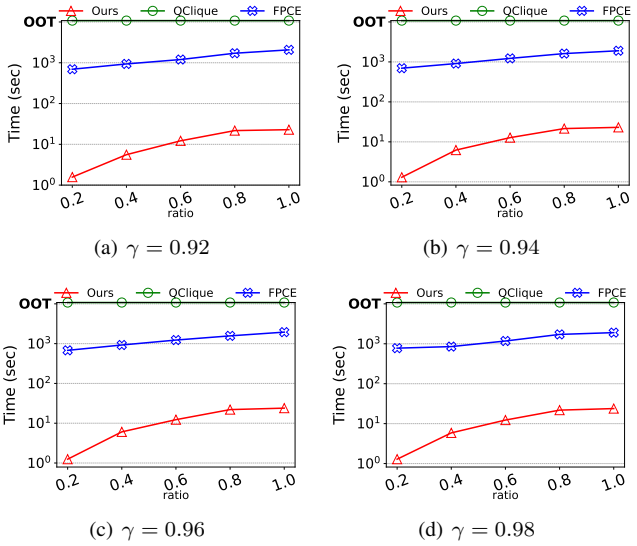
Figures 4 and 5 illustrate the number of solved instances over time on the Facebook and Real-World collections, respectively, for $\gamma \in \{0.92, 0.94, 0.96, 0.98\}$. As shown, both baseline algorithms consistently exhibit poor performance on these datasets regardless of the value of γ. On the Facebook collection, when $\gamma = 0.98$, EQC-Pro successfully solves all 114 instances, whereas the best of the two baselines solves only 9. Moreover, across all tested values of γ, EQC-Pro solves more instances within 30 seconds than either baseline does within 3 hours. For example, at $\gamma = 0.96$, EQC-Pro solves 30 instances in only 30 seconds, compared to only 3

and 7 solved by QClique and FPCE, respectively, even with a 3-hour limit. On the Real-World collection, EQC-Pro also consistently outperforms the two baseline methods under all tested values of γ. Specifically, at $\gamma = 0.94$, EQC-Pro solves 77 instances within only 30 seconds, while the best baseline solves only 68 instances even with a 3-hour limit.

Evaluation on Large-Scale Graph Instances. Table I reports the performance of the baseline algorithms QClique and FPCE, as well as our proposed method EQC-Pro (and its variants), on large-scale graphs from two dataset collections with $\gamma = 0.95$. In particular, we select 15 largest graphs from the Real-World collection and 5 largest graphs from the Facebook collection, excluding extremely difficult cases in which all three algorithms fail to solve the instance within the 3-hour time limit. The test instances are then sorted in descending order of graph size in the table. We use "–" to indicate a timeout (i.e., failure to solve within 3 hours) and highlight the best results in **bold**.

It can be observed from Table I that EQC-Pro consistently outperforms the two baselines, QClique and FPCE, across all instances. Specifically, in 13 out of 20 cases, both baselines fail to produce a solution within 3 hours, whereas EQC-Pro succeeds. For example, on the soc-delicious instance, EQC-Pro completes in 0.67 seconds, while both baselines time out, implying a speedup of at least 1.6×10^4. Furthermore, when considering the better of the two baselines, EQC-Pro achieves a speedup of over $1000\times$ in 4 instances, at least $100\times$ in 9 instances, and over $20\times$ in 14 instances.

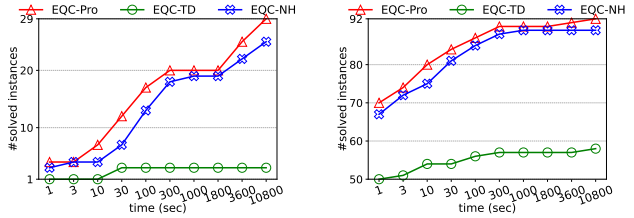
Scalability Test. To evaluate scalability, we select the largest instance across the two collections and perform subgraph sampling by randomly selecting vertices with increasing probabilities and extracting the induced subgraphs. The results are presented in Figure 6. As shown, QClique consistently fails to solve any instance within the time limit, regardless of the sampling ratio or the value of γ, indicating its inability to handle large-scale graphs.For FPCE, the runtime appears to grow nearly linearly with the number of vertices in the figure due to the use of a logarithmic scale on the y-axis. In reality, this behavior is consistent with its exponential time complexity of $O^*(2^n)$, where n is the number of vertices in the sampled subgraph. In contrast, the runtime of EQC-Pro initially increases more rapidly than that of FPCE as the graph size grows. This is because, for smaller subgraphs, the polynomial-time components of our algorithm, such as Degen-Opt and EQC-Heu-Pro, dominate the overall computational cost. However, as the graph size increases and the exponential search component becomes dominant, the advantage of our improved exponential base $O^*(\beta_\kappa^n)$ becomes increasingly apparent. Consequently, for larger graphs, EQC-Pro grows more slowly and consistently outperforms FPCE across all scales, achieving speedups of two to three orders of magnitude.



(a) $\gamma = 0.92$ (b) $\gamma = 0.94$

(c) $\gamma = 0.96$ (d) $\gamma = 0.98$

Fig. 6: Scalability test on instance socfb-konect.



(a) Facebook (b) Real-World

Fig. 7: Number of solved instances by EQC-Pro and its ablation versions (EQC-TD and EQC-NH) with $\gamma = 0.95$.

B. Ablation Study

Number of Solved Instances. Figure 7 illustrates the number of solved instances by EQC-Pro and its two ablation versions (EQC-TD and EQC-NH) under $\gamma = 0.95$. Regardless of the dataset, EQC-Pro consistently solves more instances within the same time limit across nearly all tested durations. This improvement is largely attributed to the bottom-up doubling iterative design, which not only reduces the number of iterations from $O(n)$ to $O(\log s^*)$ but also ensures that smaller values of k are used during the search, thereby making the subproblems easier to solve. Compared to the top-down variant EQC-TD, EQC-Pro exhibits substantial gains: it solves 29 and 92 instances within 3 hours on the Facebook and Real-World collections, respectively, while EQC-TD only solves 3 and 58.

In contrast, the improvement from heuristic enhancement is less significant. This is because the ablation variant EQC-NH still retains the base heuristic Degen-Opt, which in some cases already provides strong solutions. Consequently, the additional heuristic search from EQC-Heu-Pro may consume extra time without delivering benefit. Even so, it is worth noting that the time axis is plotted on a nearly exponential scale, which further highlights the efficiency of EQC-Pro: on the Facebook collection, it solves 12 instances within 30 seconds, which is comparable to the 13 instances solved by EQC-NH in 100 seconds. Similarly, on the Real-World collection, EQC-Pro solves 90 instances in 300 seconds, matching the 89 instances solved by EQC-NH in 1000 seconds.

Evaluation on Large-Scale Graph Instances. Table I reports the performance of ablation variants EQC−TD and EQC−NH on large-scale graph instances under $\gamma = 0.95$. We first consider the effect of different iterative frameworks by comparing EQC−Pro and EQC−TD. From Table I, we know that EQC−Pro, benefiting from its bottom-up doubling iterative framework, is able to solve 13 additional instances that time out under the top-down iterative method EQC−TD. In particular, on the ca−MathSciNet instance, it achieves a speedup of at least 6×10^4. For the remaining 7 instances where both methods EQC−Pro and EQC−TD complete within the time limit, the performance of EQC−TD is comparable to that of EQC−Pro. This is mainly due to the small final solution size s^* in these cases. For instance, in soc−twitter−follows and socfb−uci−uni, the maximum size of the 0.95-EQC is only 7, resulting in a limited allowance for missing edges. This tight constraint indirectly improves the accuracy of upper bound estimations, narrowing the performance gap between EQC−Pro and EQC−TD.

We next consider our proposed heuristic strategy EQC−Heu−Pro (Algorithm 6) by comparing EQC−Pro and EQC−NH. The integration of EQC−Heu−Pro also leads to a promising reduction in solving time. Among the 20 instances considered in Table I, 17 instances exhibit accelerated performance, with two additional time-out cases successfully solved by EQC−Pro. However, in certain instances, such as inf−road−usa, EQC−NH even outperforms EQC−Pro. This observation is consistent with earlier analysis: the basic heuristic method Degen−Opt, which is retained in the variant EQC−NH, is already capable of producing high-quality solutions in some scenarios. Nevertheless, for some graphs, the speedup is relatively marginal. For example, in socfb−uci−uni and socfb−konect, EQC−Pro outperforms EQC−NH by only around 5 seconds, despite total running times of approximately 28.15 seconds and 22.82 seconds, respectively. This limited speedup is primarily due to the dominance of Degen−Opt in the overall computation time on large-scale graphs, even though it runs in polynomial time. As both EQC−Pro and EQC−NH retain this component, the observed speedup is less obvious, despite a substantial reduction in the number of required iterations.

VIII. Related Work

Edge-based Quasi-Clique. The concept of edge-based γ-quasi-clique (γ-EQC) was defined in [3], [2]. This definition has also been referred to in prior work as dense subgraph [31], [47], near-clique [6], [45], [25], or pseudo clique [40]. Existing exact approaches for solving MaxEQC can be broadly categorized into two main directions: mixed integer programming (MIP) and branch-and-bound techniques. We first note that MIP-based methods [39], [34], [49] can handle graphs with at most 10^4 vertices and are difficult to scale in practice; thus, we focus on branch-and-bound algorithms. The first branch-and-bound algorithm for this problem was proposed by Pajouh et al. [37], who introduced a novel upper bounding technique and demonstrated strong performance. Later, QClique [41] incorporated a novel upper bound for pruning, which is provably tighter than previous bounds, and demonstrated that their approach is competitive with both MIP techniques [39], [49] and branch-and-bound method [37]. Although QClique achieves competitive performance, its branching scheme is identical to that of the maximal γ-EQC enumeration method PCE [48], and its pruning strategy relies on a computationally expensive upper bound. FPCE [40] also builds upon PCE and specifically addresses the maximal γ-EQC enumeration problem. FPCE introduces several refined bounding techniques, which reduces the per-branch cost from $O(n)$ to $O(\log n)$, thereby achieving state-of-the-art performance in both the maximal γ-EQC enumeration and MaxEQC problems.

Given that MaxEQC is NP-hard [39], various heuristic algorithms have been proposed for high-quality approximate solutions. Early approaches include GRASP-based methods and local search [2], [7]. Later studies explored greedy strategies [47] and, more recently, advanced vertex selection and iterative refinement techniques [13], [29].

Other Relaxed-Clique Models. As mentioned earlier, there is another type of quasi-clique defined based on vertex degree: a degree-based γ-quasi-clique requires that each vertex in the subgraph connects to at least a γ-proportion of the remaining vertices [35], [42]. DDA [38] introduced a hybrid strategy that combines MIP formulations with an iterative refinement process. More recently, Xia et al. [52] proposed IterQC, which reformulates the original problem as a series of simpler dense subgraph problems that are easier to solve and applies an iterative refinement framework to progressively obtain the optimal solution. Moreover, a closely related problem is the corresponding maximal enumeration problem, for which several branch-and-bound methods, including Quick [27] and Quick+ [24], have been developed. Most recently, Yu and Long [57] proposed the state-of-the-art method FastQC by introducing new branching and pruning strategies.

Beyond both edge-based and degree-based quasi-cliques, various other cohesive subgraph models have also been studied, including k-plex [53], [11], [51], [12], [22], k-core [4], [15], k-truss [16], [50], k-defective clique [8], [17], [9], k-VCC [28], s-bundle [30], and the densest subgraph [33], [54]. For comprehensive overviews of cohesive subgraphs, readers are referred to several excellent books and surveys, such as [10], [19], [20], [23], [26].

IX. Conclusion

In this paper, we proposed an efficient iterative algorithm EQC−Pro for solving the MaxEQC problem. By reducing MaxEQC to a series of hereditary subproblems, we improved its theoretical complexity. We further improved the algorithm with a bottom-up doubling iterative framework and introduced a novel heuristic strategy to boost its practical performance. Extensive experiments on real-world graphs showed that EQC−Pro significantly outperforms state-of-the-art algorithms in runtime, achieving up to $10^4\times$ speedup. As future work, we plan to extend EQC−Pro to the weighted setting.

REFERENCES

[1] https://github.com/SmartProbiotics/EQC-Pro, Technical Report and Source Code.

[2] J. Abello, M. G. C. Resende, and S. Sudarsky, "Massive quasi-clique detection," in *Proceedings of the Latin American Symposium on Theoretical Informatics (LATIN)*. Springer, 2002, pp. 598–612.

[3] J. Abello, P. M. Pardalos, and M. G. C. Resende, "On maximum clique problems in very large graphs," in *Proceedings of the DIMACS Workshop on External Memory Algorithms*, 1998, pp. 119–130.

[4] V. Batagelj and M. Zaveršnik, "An $O(m)$ algorithm for cores decomposition of networks," *CoRR*, vol. cs.DS/0310049, 2003.

[5] P. Bedi and C. Sharma, "Community detection in social networks," *Data Mining and Knowledge Discovery*, vol. 6, no. 3, pp. 115–135, 2016.

[6] Z. Brakerski and B. Patt-Shamir, "Distributed discovery of large near-cliques," *Distributed Computing*, vol. 24, pp. 79–89, 2011.

[7] M. Brunato, H. H. Hoos, and R. Battiti, "On effectively finding maximal quasi-cliques in graphs," in *Proceedings of the International Conference on Learning and Intelligent Optimization (LION)*, 2007, pp. 41–55.

[8] L. Chang, "Efficient maximum k-defective clique computation with improved time complexity," *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 1, no. 3, pp. 1–26, 2023.

[9] ——, "Maximum defective clique computation: Improved time complexities and practical performance," *Proceedings of the VLDB Endowment*, vol. 18, no. 2, pp. 200–212, 2024.

[10] L. Chang and L. Qin, *Cohesive Subgraph Computation over Large Sparse Graphs*. Springer Publishing Company, Incorporated, 2018.

[11] L. Chang, M. Xu, and D. Strash, "Efficient maximum k-plex computation over large sparse graphs," *Proceedings of the VLDB Endowment*, vol. 16, no. 2, pp. 127–139, 2022.

[12] L. Chang and K. Yao, "Maximum k-plex computation: Theory and practice," *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 2, no. 1, pp. 1–26, 2024.

[13] J. Chen, S. Cai, S. Pan, Y. Wang, Q. Lin, M. Zhao, and M. Yin, "NuQClq: An effective local search algorithm for maximum quasi-clique problem," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2021, pp. 12 258–12 266.

[14] X. Chen, Y. Zhou, J.-K. Hao, and M. Xiao, "Computing maximum k-defective cliques in massive graphs," *Computers & Operations Research*, vol. 127, p. 105131, 2021.

[15] J. Cheng, Y. Ke, S. Chu, and M. T. Özsu, "Efficient core decomposition in massive networks," in *Proceedings of the IEEE International Conference Data Engineering (ICDE)*, 2011, pp. 51–62.

[16] J. Cohen, "Trusses: Cohesive subgraphs for social network analysis," *National Security Agency Technical Report*, vol. 16, pp. 1–29, 2008.

[17] Q. Dai, R.-H. Li, M. Liao, and G. Wang, "Maximal defective clique enumeration," *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 1, no. 1, pp. 1–26, 2023.

[18] Q. Dai, R. Li, D. Cui, and G. Wang, "Theoretically and practically efficient maximum defective clique search," *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 2, no. 4, pp. 1–27, 2024.

[19] Y. Fang, X. Huang, L. Qin, Y. Zhang, W. Zhang, R. Cheng, and X. Lin, "A survey of community search over big graphs," *The VLDB Journal*, vol. 29, pp. 353–392, 2020.

[20] Y. Fang, K. Wang, X. Lin, and W. Zhang, "Cohesive subgraph search over big heterogeneous information networks: Applications, challenges, and solutions," in *Proceedings of the ACM on Management of Data (SIGMOD)*, 2021, pp. 2829–2838.

[21] J. Gao, Z. Xu, R. Li, and M. Yin, "An exact algorithm with new upper bounds for the maximum k-defective clique problem in massive sparse graphs," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2022, pp. 10 174 – 10 183.

[22] S. Gao, K. Yu, S. Liu, and C. Long, "Maximum k-plex search: An alternated reduction-and-bound method," *Proceedings of the VLDB Endowment*, vol. 18, p. 363–376, 2024.

[23] X. Huang, L. V. S. Lakshmanan, and J. Xu, *Community Search over Big Graphs*. Morgan & Claypool Publishers, 2019.

[24] J. Khalil, D. Yan, G. Guo, and L. Yuan, "Parallel mining of large maximal quasi-cliques," *The VLDB Journal*, vol. 31, no. 4, pp. 649–674, 2022.

[25] C. Komusiewicz, M. Sorge, and K. Stahl, "Finding connected subgraphs of fixed minimum density: Implementation and experiments," in *Proceedings of the International Symposium on Experimental Algorithms (SEA)*, 2015, pp. 82–93.

[26] V. E. Lee, N. Ruan, R. Jin, and C. Aggarwal, "A survey of algorithms for dense subgraph discovery," in *Managing and Mining Graph Data*. Springer, 2010, pp. 303–336.

[27] G. Liu and L. Wong, "Effective pruning techniques for mining quasi-cliques," in *Proceedings of the Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML/PKDD)*, 2008, pp. 33–49.

[28] H. Liu, Y. Wang, X. Xu, and D. Li, "Bottom-up k-vertex connected component enumeration by multiple expansion," in *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*, 2024, pp. 3000–3012.

[29] S. Liu, J. Zhou, and M. Lei, "An optimization algorithm for maximum quasi-clique problem based on information feedback model," *PeerJ Computer Science*, vol. 10, p. e2173, 2024.

[30] Y. Liu, H. Huang, K. Yu, S. Liu, and C. Long, "Efficient maximum s-bundle search via local vertex connectivity," *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 3, no. 1, pp. 1–27, 2025.

[31] J. Long and C. Hartman, "Odes: An overlapping dense sub-graph algorithm," *Bioinformatics*, vol. 26, no. 21, pp. 2788–2789, 2010.

[32] C. Luo, Y. Zhou, Z. Wang, and M. Xiao, "A faster branching algorithm for the maximum k-defective clique problem," in *Proceedings of the European Conference on Artificial Intelligence (ECAI)*, 2024, pp. 4132–4139.

[33] C. Ma, Y. Fang, R. Cheng, L. V. S. Lakshmanan, W. Zhang, and X. Lin, "On directed densest subgraph discovery," *ACM Transactions on Database Systems*, vol. 46, no. 4, pp. 1–45, 2021.

[34] F. Marinelli, A. Pizzuti, and F. Rossi, "LP-based dual bounds for the maximum quasi-clique problem," *Discrete Applied Mathematics*, vol. 296, pp. 118–140, 2021.

[35] H. Matsuda, T. Ishihara, and A. Hashimoto, "Classifying molecular sequences using a linkage graph with their pairwise similarities," *Theoretical Computer Science*, vol. 210, no. 2, pp. 305 – 325, 1999.

[36] T. Nepusz, H. Yu, and A. Paccanaro, "Detecting overlapping protein complexes in protein–protein interaction networks," *Nature Methods*, vol. 9, no. 5, pp. 471–472, 2012.

[37] F. M. Pajouh, Z. Miao, and B. Balasundaram, "A branch-and-bound approach for maximum quasi-cliques," *Annals of Operations Research*, vol. 216, no. 1, pp. 145–161, 2014.

[38] G. Pastukhov, A. Veremyev, V. Boginski, and O. A. Prokopyev, "On maximum degree-based γ-quasi-clique problem: Complexity and exact approaches," *Networks*, vol. 71, no. 2, pp. 136–152, 2018.

[39] J. Pattillo, A. Veremyev, S. Butenko, and V. Boginski, "On the maximum quasi-clique problem," *Discrete Applied Mathematics*, vol. 161, no. 1-2, pp. 244–257, 2013.

[40] A. Rahman, K. Roy, R. Maliha, and T. F. Chowdhury, "A fast exact algorithm to enumerate maximal pseudo-cliques in large sparse graphs," in *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (SIGKDD)*, 2024, pp. 2479–2490.

[41] C. Ribeiro and J. R. Merino, "An exact algorithm for the maximum quasi-clique problem," *International Transactions in Operational Research*, vol. 26, no. 1, pp. 129–151, 2019.

[42] S.-V. Sanei-Mehri, A. Das, H. Hashemi, and S. Tirthapura, "Mining largest maximal quasi-cliques," *ACM Transactions on Knowledge Discovery from Data*, vol. 15, pp. 81:1–81:21, 2021.

[43] S. B. Seidman, "Network structure and minimum degree," *Social Networks*, vol. 5, no. 3, pp. 269–287, 1983.

[44] S. B. Seidman and B. L. Foster, "A graph-theoretic generalization of the clique concept," *Journal of Mathematical Sociology*, vol. 6, no. 1, pp. 139–154, 1978.

[45] S. Tadaka and K. Kinoshita, "Ncmine: Core-peripheral based functional module detection using near-clique mining," *Bioinformatics*, vol. 32, no. 22, pp. 3454–3460, 2016.

[46] B. K. Tanner, G. Warner, H. Stern, and S. Olechowski, "Koobface: The evolution of the social botnet," in *2010 eCrime Researchers Summit*, 2010, pp. 1–10.

[47] C. E. Tsourakakis, F. Bonchi, A. Gionis, F. Gullo, and M. A. Tsiarli, "Denser than the densest subgraph: Extracting optimal quasi-cliques with quality guarantees," in *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (SIGKDD)*, 2013, pp. 104–112.

[48] T. Uno, "An efficient algorithm for solving pseudo clique enumeration problem," *Algorithmica*, vol. 56, no. 1, pp. 3–16, 2010.

[49] A. Veremyev, O. A. Prokopyev, S. Butenko, and E. L. Pasiliao, "Exact mip-based approaches for finding maximum quasi-cliques and dense

subgraphs," *Computational Optimization and Applications*, vol. 64, no. 1, pp. 177–214, 2016.
[50] J. Wang and J. Cheng, "Truss decomposition in massive networks," *Proceedings of the VLDB Endowment*, vol. 5, no. 9, pp. 812–823, 2012.
[51] Z. Wang, Y. Zhou, C. Luo, and M. Xiao, "A fast maximum k-plex algorithm parameterized by the degeneracy gap," in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2023, pp. 5648–5656.
[52] H. Xia, K. Yu, S. Liu, C. Long, and X. Zhou, "Maximum degree-based quasi-clique search via an iterative framework," in *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (SIGKDD)*, 2025.
[53] M. Xiao, W. Lin, Y. Dai, and Y. Zeng, "A fast algorithm to compute maximum k-plexes in social network analysis," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2017, pp. 919–925.
[54] Y. Xu, C. Ma, Y. Fang, and Z. Bao, "Efficient and effective algorithms for densest subgraph discovery and maintenance," *The VLDB Journal*, vol. 33, no. 5, pp. 1427–1452, 2024.
[55] H. Yu, A. Paccanaro, V. Trifonov, and M. Gerstein, "Predicting interactions in protein networks by completing defective cliques," *Bioinformatics*, vol. 22, no. 7, pp. 823–829, 2006.
[56] K. Yu and C. Long, "Graph mining meets fake news detection," in *Data Science for Fake News*, 2021, pp. 169–189.
[57] ——, "Fast maximal quasi-clique enumeration: A pruning and branching co-design approach," *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 1, no. 3, pp. 1–26, 2023.

APPENDIX

A. Correctness Proof of Algorithm 1

We next prove the correctness of our top-down iterative framework EQC-TD for solving MaxEQC in Algorithm 1. Before that, we first establish two supporting lemmas.

Lemma 1. *Recall that s^* denotes the size of the maximum edge-based γ-quasi-clique. If a value s satisfies the condition $s = \texttt{solve-defect}(\texttt{get-k}(s))$, then we have $s \leq s^*$.*

Proof. Let

$$k = \texttt{get-k}(s) = \left\lfloor \binom{s}{2} \times (1-\gamma) \right\rfloor.$$

Since $\texttt{solve-defect}(k) = s$, there exists a subgraph g of G with $|V(g)| = s$ that is a k-defective clique. Thus, we have

$$|E(g)| \geq \binom{s}{2} - k \geq \binom{s}{2} - \binom{s}{2}(1-\gamma) = \binom{s}{2}\gamma,$$

which implies that g is an edge-based γ-quasi-clique of size s. By the definition of s^* as the maximum size of any edge-based γ-quasi-clique, we conclude that $s \leq s^*$, as desired. □

Lemma 2. *Recall that s^* is the size of the maximum edge-based γ-quasi-clique. Let $ub > s^*$ be an upper bound before an iteration (i.e., an execution of the while-loop in Lines 2-6 of Algorithm 3), where ub may be the s value obtained in the last iteration. Define $k = \texttt{get-k}(ub)$ and ub' as the s value after the iteration, i.e., $ub' = \texttt{solve-defect}(k)$. Then, we have $s^* \leq ub' < ub$. In other words, ub' is a strictly tighter upper bound on s^*.*

Proof. We observe that both get-k and solve-defect functions are non-decreasing. By Proposition 2, we know that $\texttt{solve-defect}(\texttt{get-k}(s^*)) \geq s^*$. Applying this to $ub > s^*$, we obtain

$$\begin{aligned} ub' &= \texttt{solve-defect}(\texttt{get-k}(ub)) \\ &\geq \texttt{solve-defect}(\texttt{get-k}(s^*)) \geq s^*. \end{aligned}$$

On the other hand, since $ub > s^*$ and with Proposition 2, we have

$$ub' = \texttt{solve-defect}(\texttt{get-k}(ub)) < ub.$$

Combining the two bounds yields the relation $s^* \leq ub' < ub$, which completes our proof. □

Proposition 3. *Algorithm 1 can correctly computes the size of the maximum edge-based γ-quasi-clique in finite steps.*

Proof. By Lemma 1, whenever the termination condition of the while-loop (Line 4) is satisfied, the returned value satisfies $s_i \leq s^*$. By Lemma 2, as long as the termination condition is not met, the sequence $\{s_0, s_1, \ldots, s_i\}$ is strictly decreasing and remains an upper bound on s^*. Further, the s value at each iteration satisfies $s_i \geq s^*$ until the termination condition is triggered. Thus, the only possibility at termination is that $s_i = s^*$ and the algorithm can be terminated in finite steps. □

Algorithm 8: ComputeUB(k) [8], [18]

Output: An upper bound of the size of the maximum k-defective clique.

colorCount[] ← GreadyColor(G);
Let numColors be the total number of colors used;
maxCount ← 0;
foreach $c = 1$ to *numColors* **do**
  maxCount ← max(maxCount, colorCount[c]);
Perform a binary search over $l \in [0, \text{maxCount}]$ to find the largest l such that:

$$\sum_{c=1}^{\text{numColors}} \binom{m_c}{2} \leq k, \text{ where } m_c = \min(l, \text{colorCount}[c]);$$

$ub \leftarrow 0$;
foreach $c = 1$ to *numColors* **do**
  $m_c \leftarrow \min(l, \text{colorCount}[c])$;
  $k \leftarrow k - \binom{m_c}{2}$;
  $ub \leftarrow ub + m_c$;
$ub \leftarrow ub + \lfloor k/(l+1) \rfloor$;
return ub;
Function GreedyColor(G)**:**
  foreach *vertex u in reverse degeneracy order* **do**
    Assign to u the smallest color that has not been assigned to any of its colored neighbors;
  return an array containing the number of vertices assigned to each color;

B. Omitted Details of Our Upper Bound Computation

We first present the existing ComputeUB algorithm [8], [18] for computing an upper bound of the size of the maximum k-defective clique, which is used in our upper bound computation algorithm UpperBound (Algorithm 2). We then prove that in our upper bound computation algorithm UpperBound, the iterative procedure (Lines 2-5 of Algorithm 2) produces a strictly decreasing sequence of upper bounds, each of which remains a valid upper bound of the optimal value s^*, i.e., the size of the maximum γ-EQC. This result can also be used to demonstrates the correctness of our UpperBound.

ComputeUB algorithm. The key idea of ComputeUB is to apply a vertex coloring strategy that assigns different colors to adjacent vertices, partitioning the graph into independent sets (i.e., each color class consists non-adjacent vertices). ComputeUB focuses only on missing edges within each color class. During the selection process, vertices are greedily added such that each new vertex introduces the fewest missing edges among already selected vertices of the same color. This process continues until the total number of missing edges reaches or exceeds the threshold k. Since missing edges between different color classes are ignored, the number of selected vertices provides an upper bound on the size of the maximum k-defective clique. We remark that since the greedy selection is based on a fixed coloring order rather than k, ComputeUB is monotonic

in k: for $k' > k$, $\texttt{ComputeUB}(k') \geq \texttt{ComputeUB}(k)$.

ComputeUB is shown in Algorithm 8. The graph is first greedily colored in Line 1. Then, vertices are selected greedily to minimize new missing edges within each color class. To improve the efficiency, we use a batch-wise selection strategy: initially, up to l vertices per color class are selected, followed by at most one more vertex per class. Since the coloring order is fixed, l can be efficiently found via binary search (Lines 2-6), after which any remaining selections are made iteratively (Lines 7–12). Note that as long as the coloring order is fixed, the correctness is guaranteed. In our implementation, we use the reverse degeneracy ordering for coloring.

Correctness of Our Proposed `UpperBound`. We first note that ComputeUB (Algorithm 8) is monotone, i.e., for any $k_1 \geq k_2$, we have $\texttt{ComputeUB}(k_1) \geq \texttt{ComputeUB}(k_2)$. Based on this monotonicity, we present the following lemma.

Lemma 3. *During the iterative process (Lines 2-5) of Algorithm 2, the computed upper bound ub forms a non-increasing sequence and eventually terminates at a value that is an upper bound of s^*, i.e., the size of the maximum γ-EQC.*

Proof. Algorithm 2 iteratively computes a sequence of upper bounds on the size of the optimal solution s^*. We prove that upon termination, the final upper bound is a valid upper bound on s^*, and that the algorithm terminates in finitely many steps.

Initially, in Line 1 of Algorithm 2, the value of k is set by assuming that the size of the optimal solution is ub. Specifically, the algorithm computes the maximum number of missing edges allowed in a γ-EQC of size ub, which serves as an upper bound on the number of missing edges in the corresponding maximum k^*-defective clique, i.e., $k^* = \texttt{get-k}(s^*)$.

Consider two consecutive iterations. Let ub_1 denote the current upper bound of the optimal value s^* at the beginning of an iteration, and let k_1 be the corresponding upper bound on the number of missing edges, i.e., $k_1 \geq k^*$. We define $k_2 = \texttt{get-k}(ub_1)$ as the maximum number of missing edges associated with a γ-EQC of size ub_1, and let $ub_2 = \texttt{ComputeUB}(k_2)$ be the new upper bound computed for the next iteration. In each subsequent iteration, the algorithm updates $ub_1 \leftarrow ub_2$ and $k_1 \leftarrow k_2$. Observe that the while loop proceeds only if $k_2 = \texttt{get-k}(ub_1) < k_1$, ensuring that the value of k strictly decreases with each iteration. Since the function ComputeUB is monotonic with respect to k, we have: $ub_2 = \texttt{ComputeUB}(k_2) \leq \texttt{ComputeUB}(k_1) = ub_1$, and hence, the sequence of upper bounds is non-increasing.

Furthermore, ComputeUB always returns a valid upper bound on the size of the maximum k-defective clique for the given k. By construction, $k_2 = \texttt{get-k}(ub_1) \geq \texttt{get-k}(s^*) = k^*$, and thus $ub_2 = \texttt{ComputeUB}(k_2) \geq \texttt{ComputeUB}(k^*) = s^*$ throughout the process. Since k is a non-negative integer and can only take finitely many values bounded below by k^*, and since k strictly decreases in each iteration, the algorithm must terminate after finitely many steps. Upon termination, the final ub remains a valid upper bound on s^*. □

C. Omitted Example

An Example of `EQC-Heu`. We use Figure 2 to illustrate the heuristic procedure described in Algorithm 5. Assume that the heuristic solution from the previous algorithm is of size 2, that is, $V(g) = \{1, 4\}$. Before each iteration, EQC-Heu selects each vertex in S as a seed and performs vertex addition and deletion operations accordingly.

Consider the case where vertex 4 is chosen as the seed. In the first iteration, during the addition phase, one vertex from its neighbors $\{1, 3, 5\}$ is selected. If vertex 5 is chosen, then the next vertex to be added to S is deterministically vertex 6, which has the highest number of connections to the current S. This favorable outcome leads directly toward the optimal solution. In a less ideal scenario, suppose vertex 1 is selected instead. In the second addition step, a vertex from $\{2, 3, 5\}$ is randomly chosen. Assume vertex 5 is selected. During the deletion phase, EQC-Heu selects one vertex from $\{1, 5\}$ for removal. According to Line 19 of the algorithm, vertex 4 is excluded from deletion. Since both vertices 1 and 5 have equal scores of 0, one of them is removed at random. Suppose vertex 1 is removed. After deletion, vertex scores are updated: vertices 1 and 3 are assigned a score of 1, and vertex 6 a score of 2, based on their connections to the updated set S. Given that $d_G^C(4) = 3$, $d_G^C(5) = 1$, and the maximum degree $\Delta_G = 4$, the scores of vertices 4 and 5, which are already included in the current solution set S, are updated to -1 and -3, respectively.

In the second iteration, the first vertex added is again deterministically vertex 6, as it has the highest degree in S, which means $d_G^S(\cdot) = 2$. The second vertex added, which is either 1 or 3, will ultimately be removed because it becomes the unique lowest-degree vertex in the updated set S. At this point, the set S satisfies the condition in Line 5, and a better solution is successfully updated.